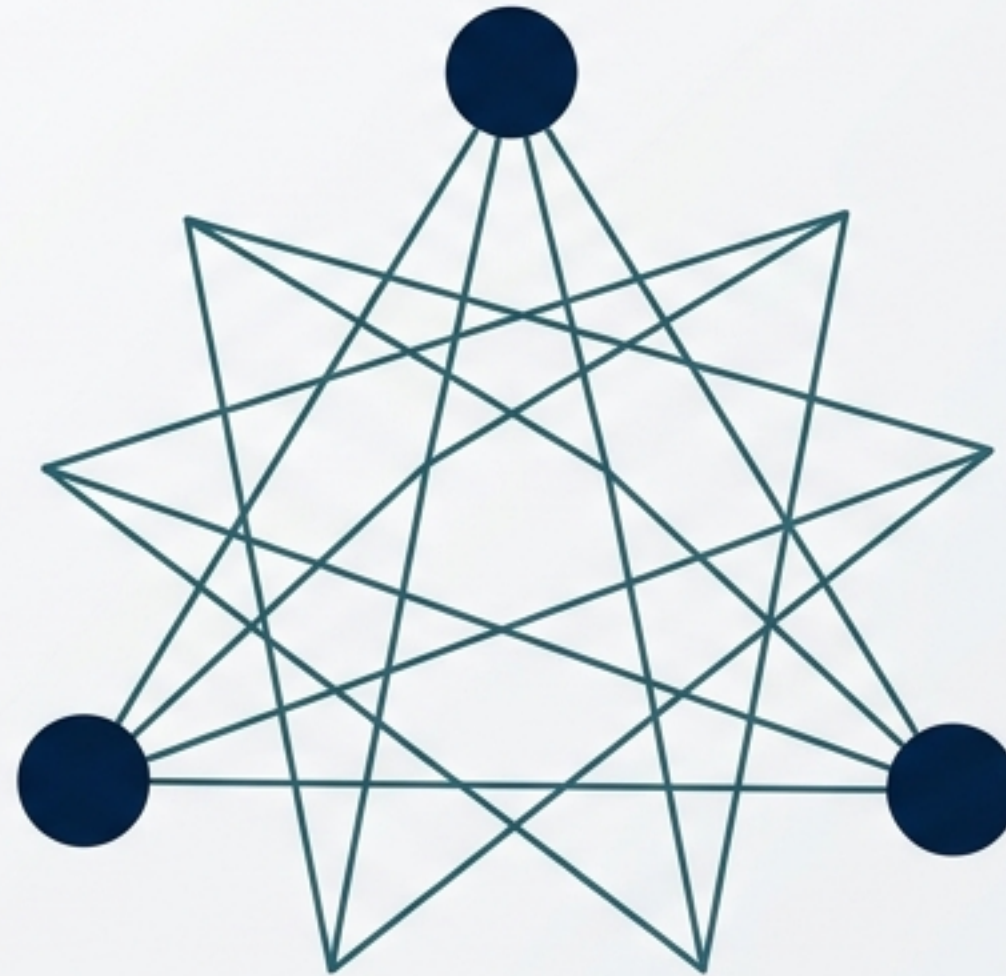


Communication Paradigms in Distributed Systems

From Direct Invocation to Decoupled Architectures



The Power of Indirection

“All problems in computer science can be solved by another level of indirection.”

– Cambridge Researchers

Indirect communication removes direct coupling between sender and receiver, solving scaling and performance bottlenecks in distributed systems.

The Dimensions of Decoupling

Time Coupling (Coupled → Uncoupled)

Must sender and receiver exist at the same moment?

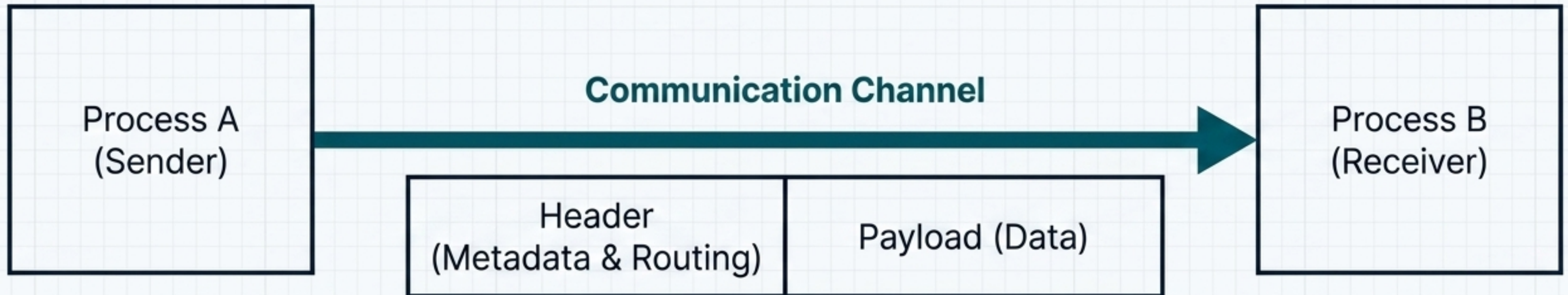
Space Coupling (Coupled → Uncoupled)

Does the sender need to know the receiver's identity?

Space & Time Coupled (RPC, Message Passing)	Space Coupled, Time Uncoupled (Email)
Space Uncoupled, Time Coupled (IP Multicast)	Space & Time Uncoupled (Pub-Sub, Message Queues)

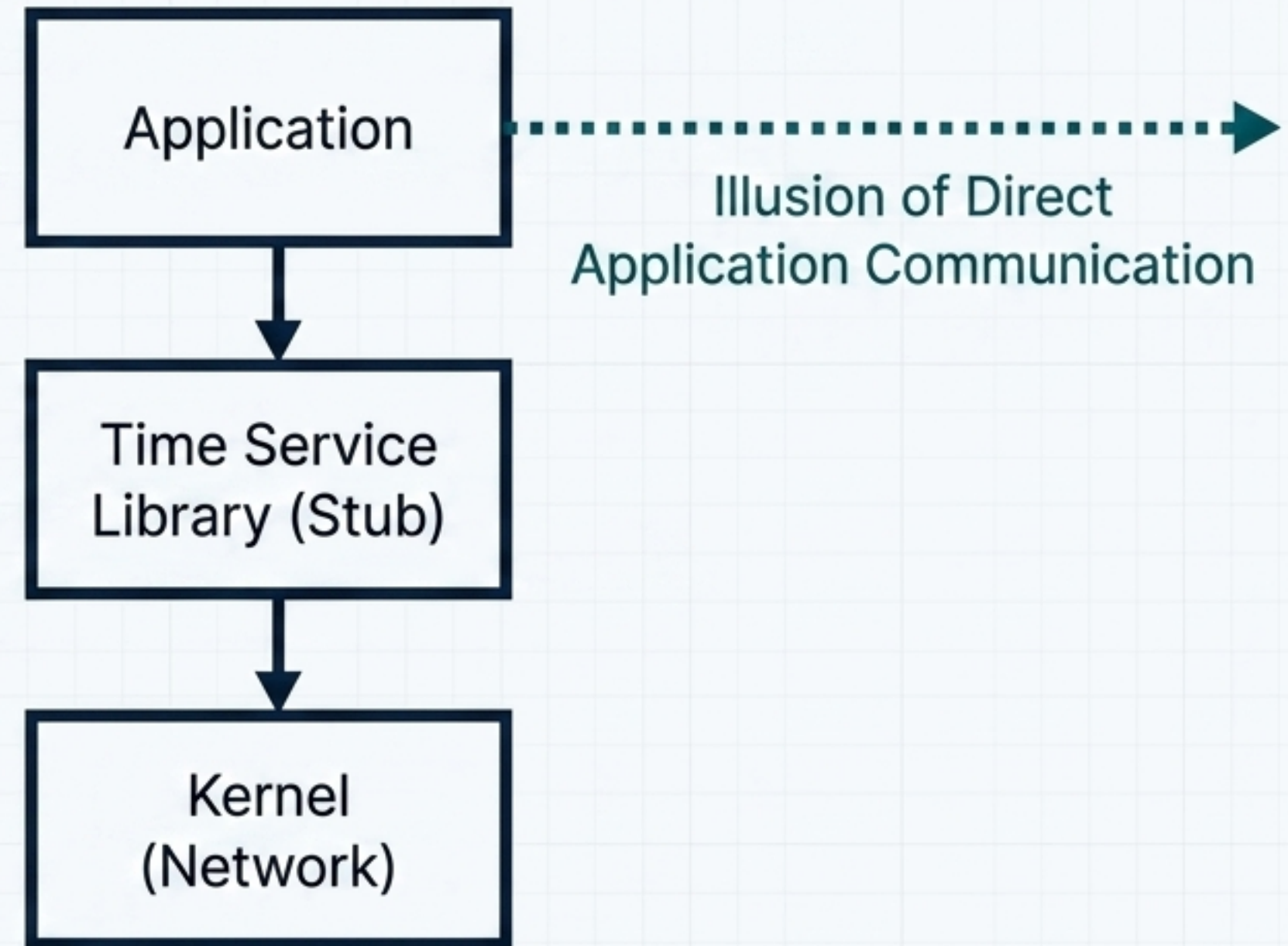
Base Primitives of Message Passing

- The foundation of Interprocess Communication (IPC).
- Relies on explicit, targeted send() and receive() system calls.
- Requires direct synchronization (blocking or non-blocking).



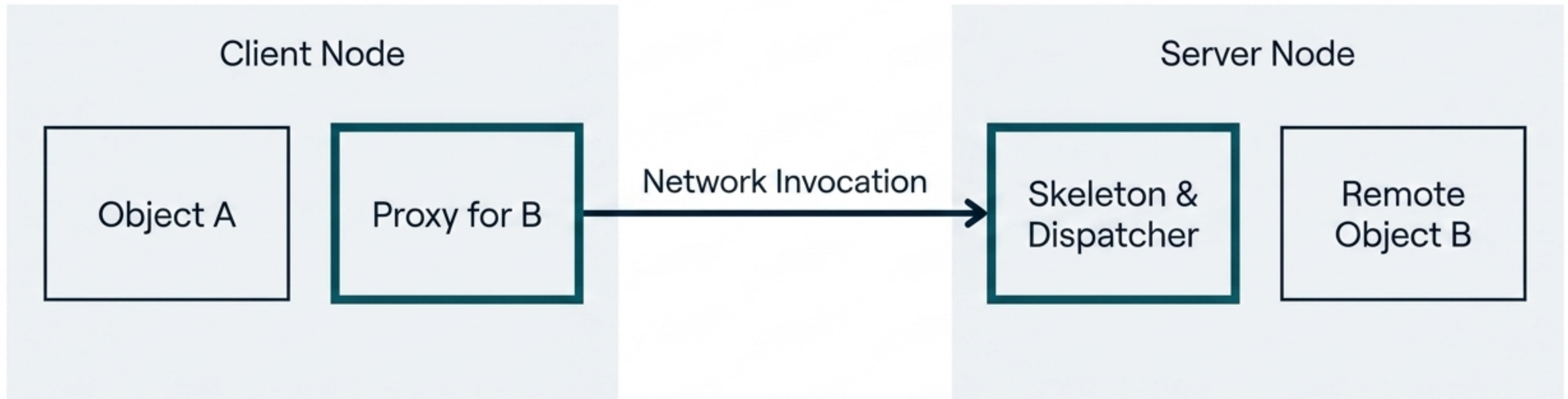
Remote Procedure Call (RPC)

- Goal: Make remote calls look identical to local calls.
- Hides network complexity, parameter encoding, and message passing.
- Vulnerability: Retains tight client-server coupling; latency and network failures are abstracted but still exist.



Remote Method Invocation Architecture

- **Proxy:** Resides on the client, behaving exactly like the remote object.
- **Skeleton & Dispatcher:** Resides on the server, unmarshals data, and invokes the true method.



Managing Distributed Object Lifecycles

Remote Reference Module:

Translates between local and remote object references.

Distributed Garbage Collection:

Servers actively track which remote processes hold references to their objects.

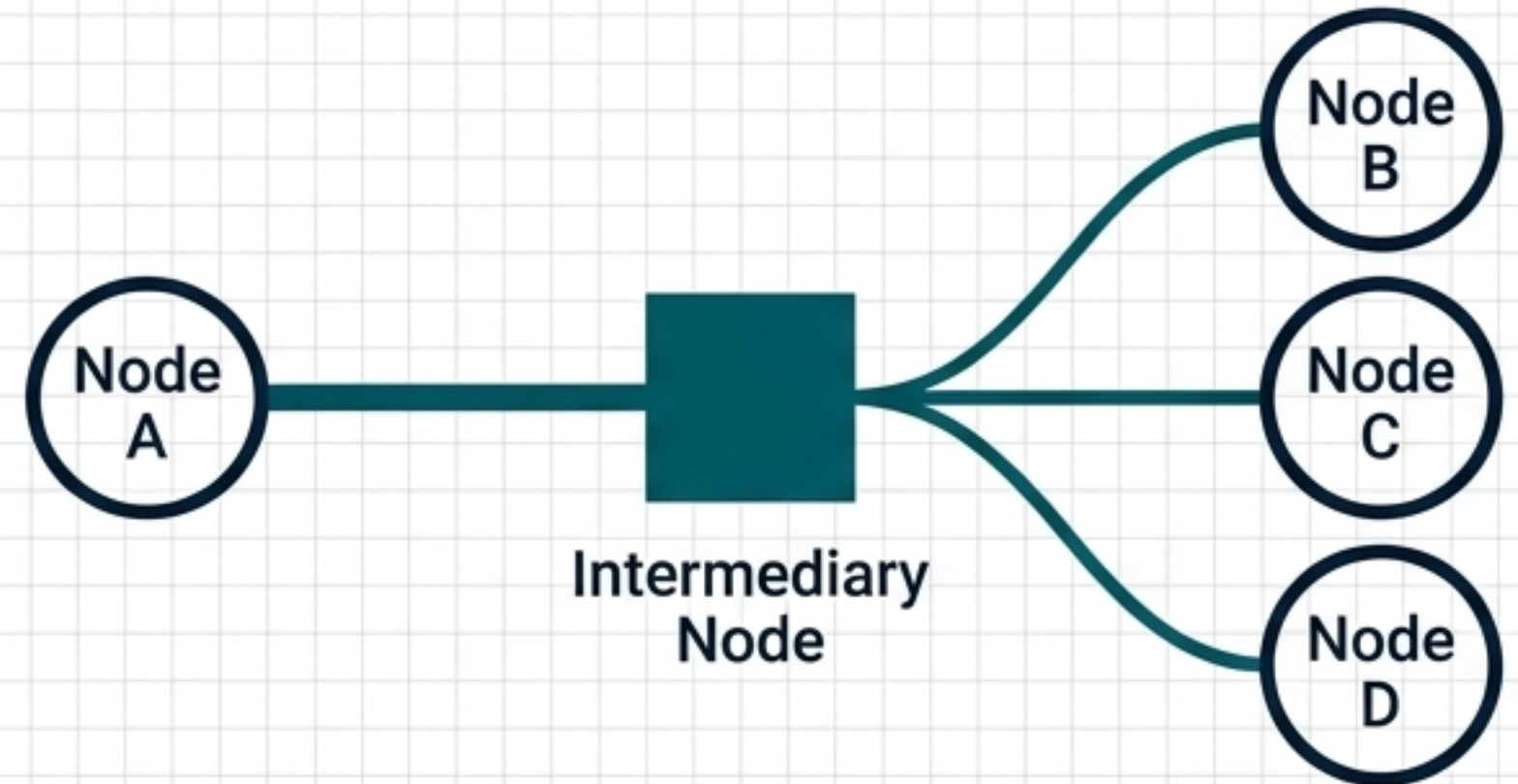
Exception Handling:

Must explicitly catch network failures distinctly from local application errors.

RMI extends object-oriented programming across networks, but requires sophisticated memory tracking to prevent leaks.

The Limits of Direct Coupling

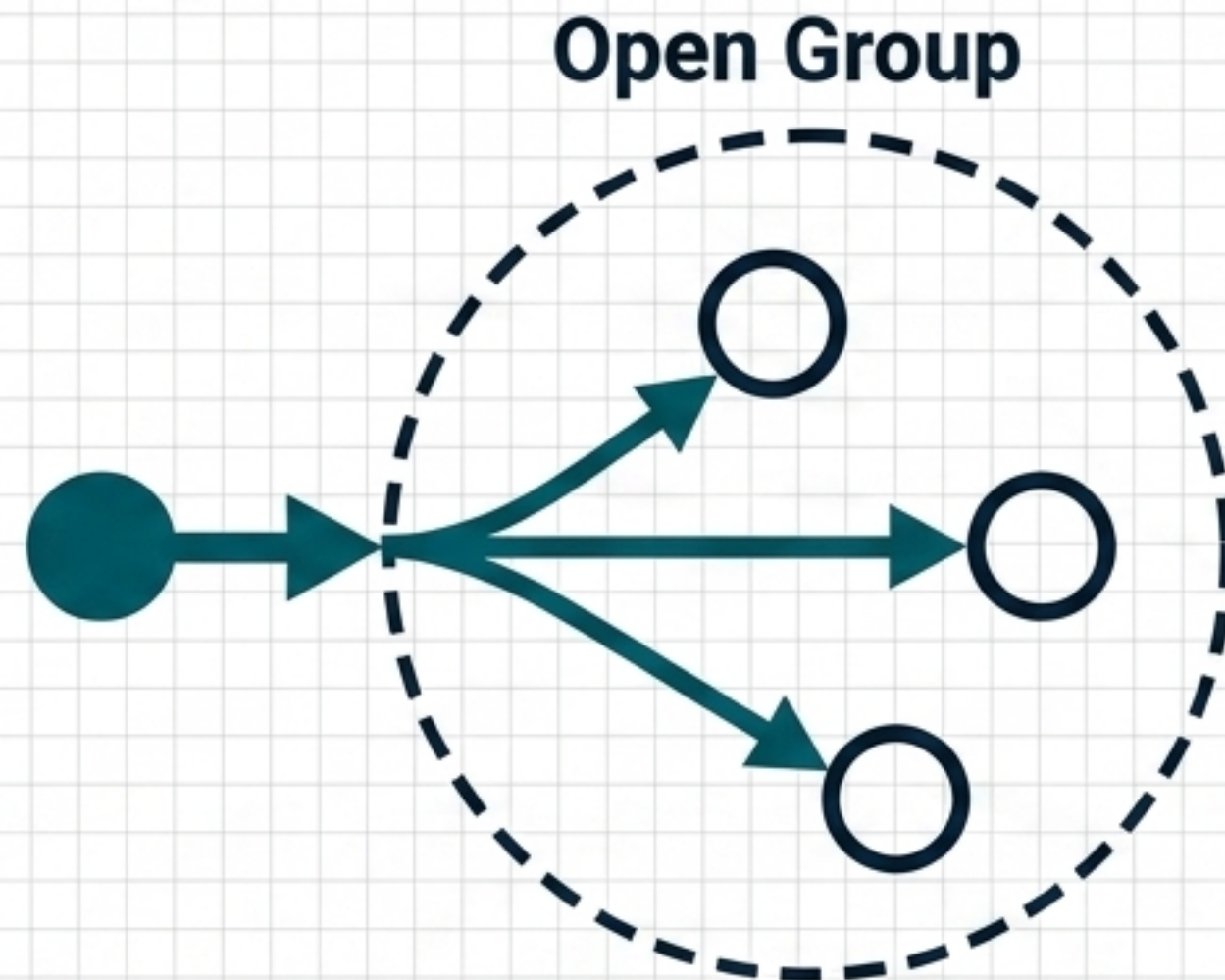
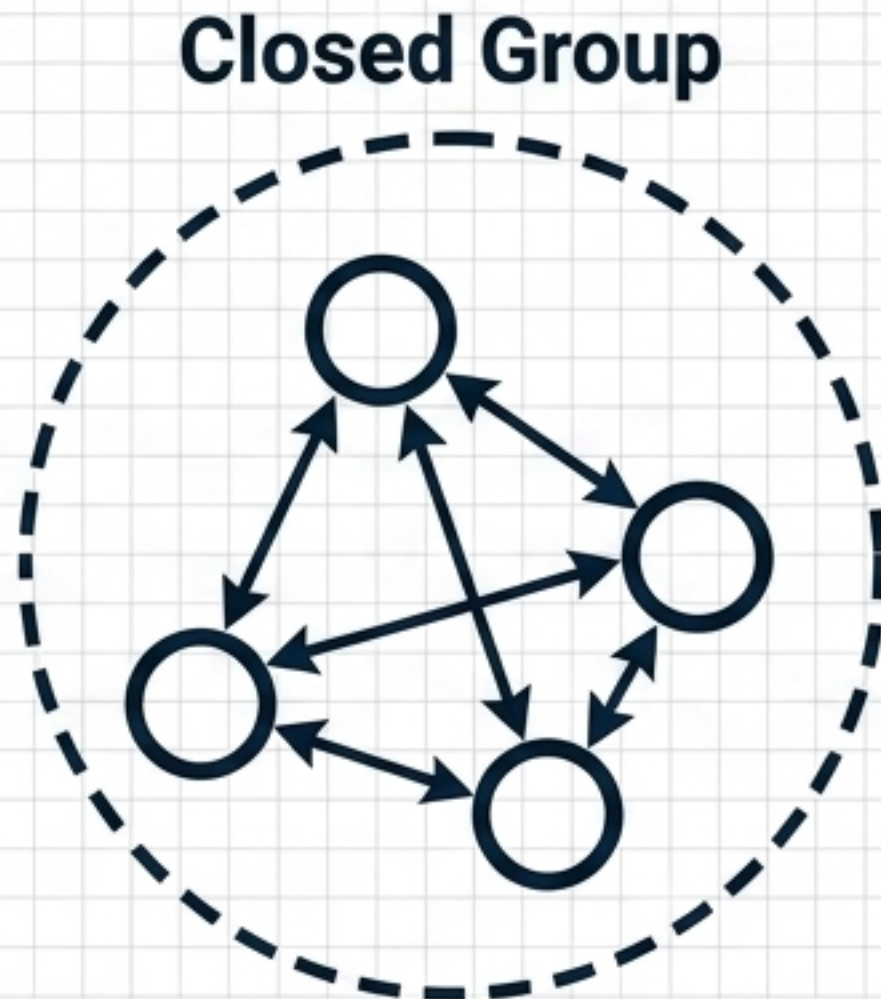
- Direct client-server models are brittle.
- Hard to replace servers transparently.
- Nearly impossible to disseminate data to dynamic, unknown groups.



Solution: Introduce an intermediary to decouple the sender from the receiver.

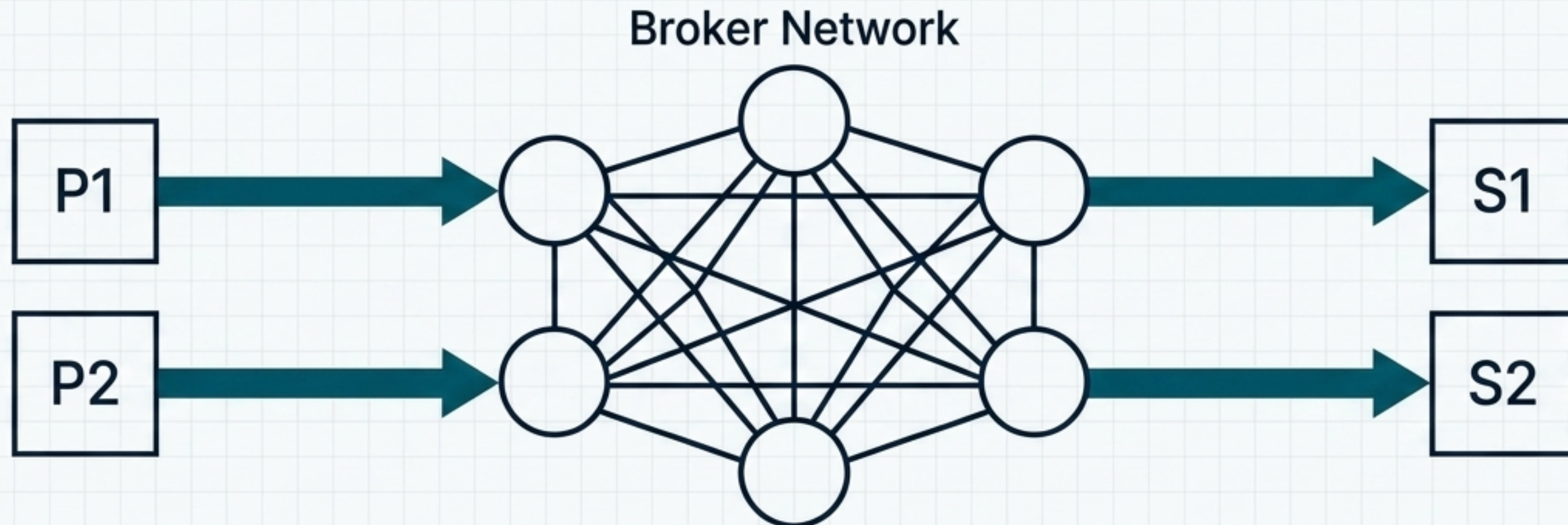
Group Communication Paradigms

- Single message sent to a group endpoint; delivered to all members.
- Provides massive value: Failure detection, membership management, and consistent ordering.
- Ideal for highly available replicated servers.



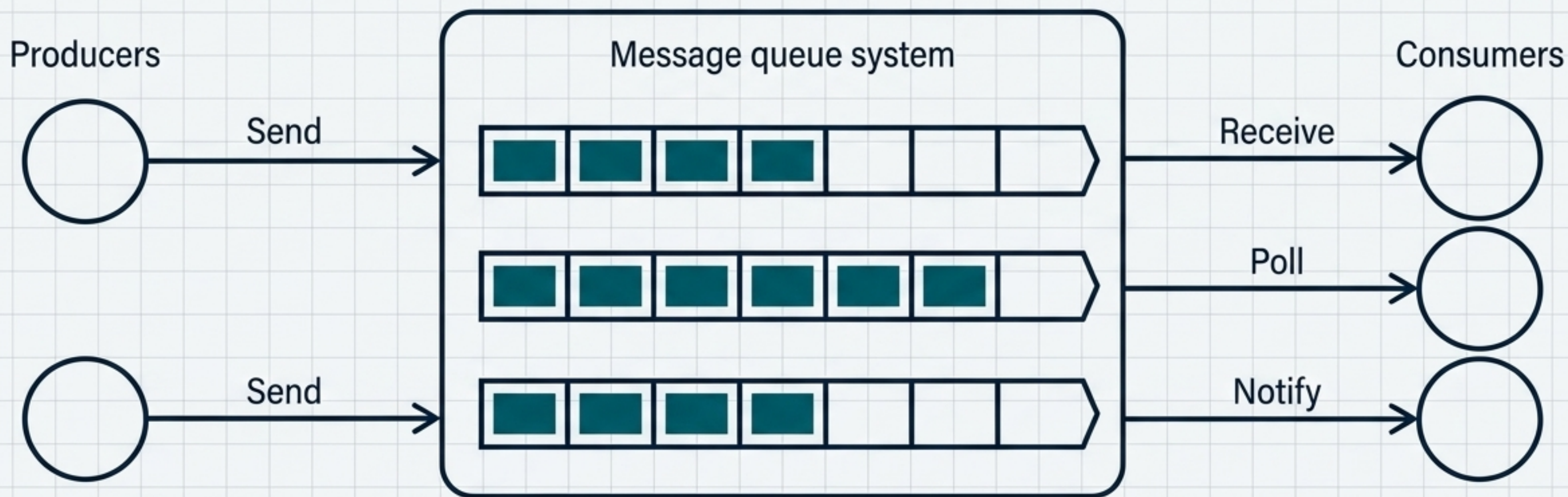
Publish-Subscribe Event Routing

- Publishers emit structured events; Subscribers express interest.
- Brokers dynamically match events to subscriptions.
- The dominant paradigm for scalable, real-time heterogeneous systems.



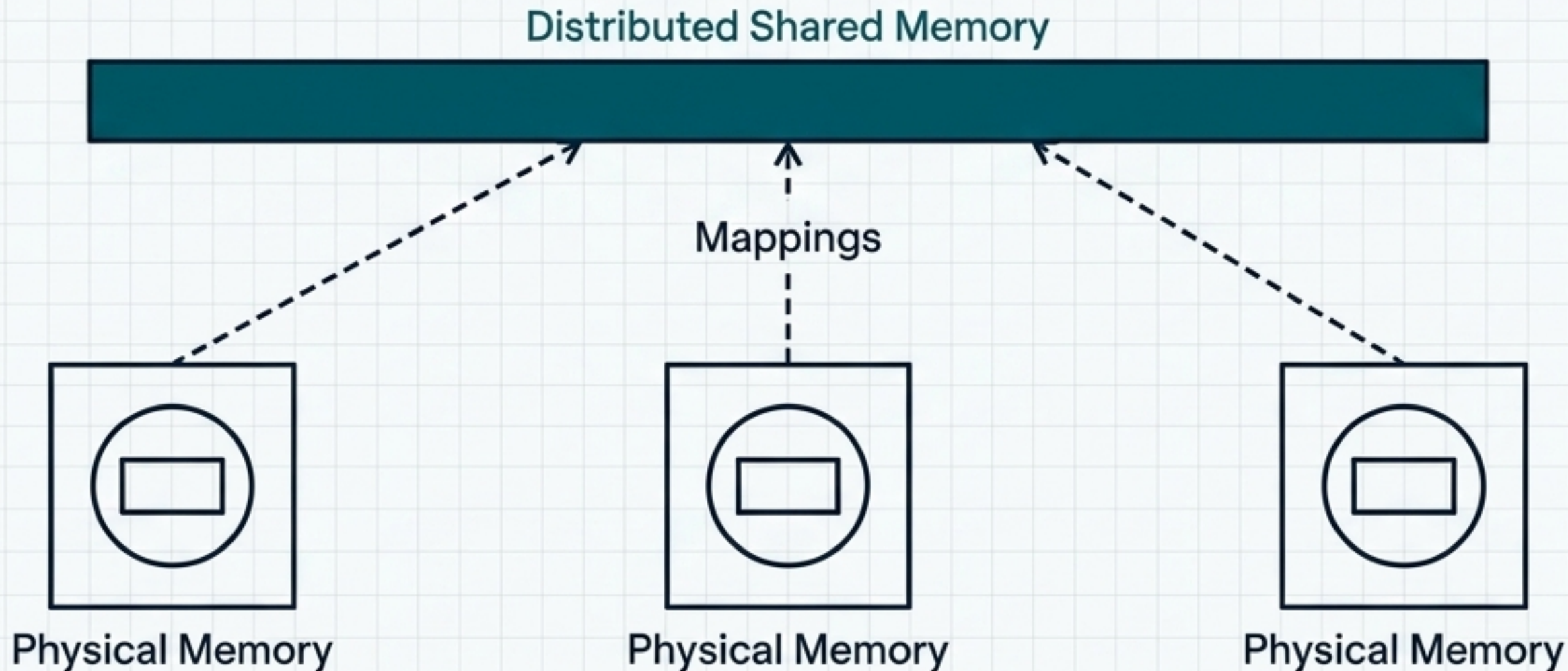
Message-Oriented Middleware (MOM)

- Point-to-point, asynchronous communication.
- Messages are persistent (stored safely on disk until removed).
- Critical for Enterprise Application Integration (EAI) and strict transactions.



Distributed Shared Memory (DSM)

- Illusion of a single shared memory space over physical network silos.
- State-based interaction rather than message-based.
- Primarily utilized for parallel computations, not standard client-server apps.



Tuple Space Communication

- A “Tuple” is an ordered list of typed values.
- A “Tuple Space” is an unordered bag of these tuples.
- Processes interact by placing tuples in the space or withdrawing them based on pattern matching.

[value, value, value]

Provides both **time** and **space uncoupling** using **content-addressable** state.

Indirect Paradigms Compared

Paradigm	Coupling	Pattern	Intent
Groups	Space Uncoupled	1-to-many	Reliability
Pub-Sub	Space & Time Uncoupled	1-to-many	Event Dissemination
Msg Queues	Space & Time Uncoupled	1-to-1	Enterprise Trans.
DSM	Space & Time Uncoupled	1-to-many	Parallel Compute

Architectural Takeaways

- **Direct invocation (RPC/RMI) simplifies remote calls but retains brittle system coupling.**
- **Space uncoupling (Groups) removes the need to strictly map network endpoints.**
- **Time uncoupling (Queues/Pub-Sub) allows systems to interact safely across different lifespans and speeds.**
- **The choice of paradigm depends entirely on the required synchronization, transaction strictness, and network topology.**